POLYMORPHISM
ENCAPSULATION
INHERITANCE
ABSTRACTION
POLYMORPHISM
ABSTRACTION
INHERITANCE
ENCAPSULATION
POLYMORPHISM
INHERITANCE
ENCAPSULATION
ABSTRACTION
POLYMORPHISM

eMódulo

Welcome to eMódulo

Email

Enter Email

Create Password

Enter Password

Confirm Password

Enter Password

Sign up

Already have an account?

eMódulo

Email

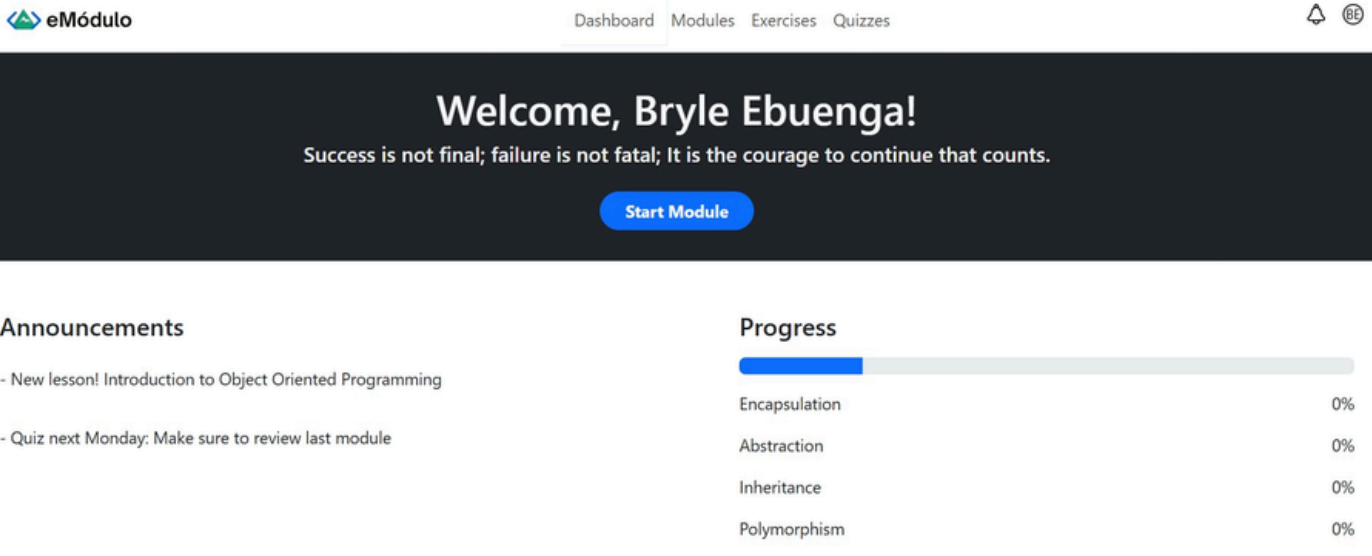
Enter Email

Password

Enter Password

Log in

Sign up


eMódulo
Dashboard
Modules
Exercises
Quizzes
BE
Welcome, Bryle Ebuenga!
Success is not final; failure is not fatal; It is the courage to continue that counts.
Start Module
Announcements
- New lesson! Introduction to Object Oriented Programming
- Quiz next Monday: Make sure to review last module
Progress
Encapsulation 0%
Abstraction 0%
Inheritance 0%
Polymorphism 0%

Welcome, Student # 1

Success is not final; failure is not fatal; It is the courage to continue that counts.

Start Module

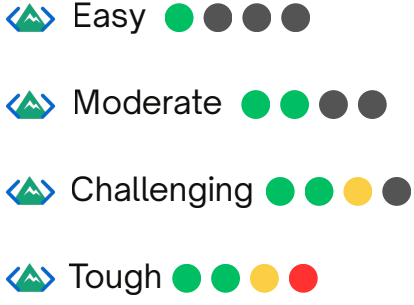
Announcements

- New lesson! Introduction to Object-Oriented Programming
- Quiz next Monday: Make sure to review the last module

Progress

Encapsulation 0%
Abstraction
Inheritance
Polymorphism

Select level of Difficulties:



Next

Easy

Proceed to Pre Test

- **System behavior**
  - Presents basic recall - type questions (definition, simple code identification).
  - Provides unlimited hints, no timer.
  - No lives
  - In order to receive your certificate you must achieve all badges and finish the module.
- **Expected Outcomes**
  - Builds confidence for beginner.
  - Encourages participation of learner.
  - Students experience early success.

Moderate

Proceed to Pre Test

- **System behavior**
  - Introduces application-type problems (trace outputs, apply OOP Concepts in code small snippets.
  - Timer enabled
  - Hints are available, but using them reduces points.
  - In order to receive your certificate you must achieve all badges and finish the module.
- **Expected Outcomes**
  - Students starts to face real challenges.
  - Promotes problem-solving skills and decision making (“Should I use a hint or try harder?”).
  - Motivation comes from overcoming slightly harder but achievable goals.

Challenging

Proceed to Pre Test

- **System behavior**
  - Focuses on problem-solving + debugging tasks (find the error, predict program output, apply inheritance/polymorphism).
  - Limited hints (costs lives or points to use them)
  - Strict timers to simulate coding pressure.
  - In order to receive your certificate you must achieve all badges and finish the module.
- **Expected Outcomes**
  - Learners experience a “flow state” (tasks are tough but rewarding).
  - Fosters grit and persistence: failure forces retry, but each retry builds competence.
  - Students who pass this stage feel a sense of achievement.

Tough

Proceed to Pre Test

- **System behavior**
  - Coding challenges or case-based problems aligned with syllabus.
  - Very strict timer + very limited lives.
  - No hints (or extremely costly).
  - Rewards = highest badges (Mastery, Challenger, Persistence Awards).
  - In order to receive your certificate you must achieve all badges and finish the module.
- **Expected Outcomes**
  - Learners demonstrate mastery and resilience.
  - Students realize they can conquer the “hardest mountain” after persistence.
  - Aligns with challenge-based intrinsic motivation → the hardest path gives the most satisfying growth.

Pre test Easy

- This pre-test contains 5 multiple-choice questions on the basics of Object-Oriented Programming (OOP).
- Read each question carefully and select the best answer.
- You may use the hint button if needed (no penalty in Easy Level).
- There is no strict timer—take your time to understand the questions.
- Each correct answer is worth 1 point.
- At the end, you will receive feedback and a badge based on your score.
- Completion of this pre-test is required, and you will earn the “Starter Challenger” badge.

Next

Are you ready?

Proceed

Easy

1.Which of the following correctly defines a class in OOP?
a. A loop that repeats code
b. A blueprint for creating objects
c. A method of abstraction
d. A library of functions

Hint

Answer: b. A blueprint for creating objects

Remember: Classes define the structure and behavior of objects.

2.In OOP, what do we call the real-world entity created from a class?
a. Method
b. Object
c. Variable
d. Function

Hint

Answer: b.

Remember: Objects are instances of classes, representing real-world entities.

Next

Easy

3.Which of these is NOT an OOP principle?
a. Inheritance
b. Encapsulation
c. Polymorphism
d. Compilation

Hint

Answer: d. Compilation

Remember: Compilation is part of program execution, not an OOP concept.

4.What does Encapsulation mean in OOP?
a. Wrapping data and methods together
b. Using many functions and once
c. Breaking code into similar parts
d. Writing loops inside a class

Hint

Answer: a. Wrapping data and methods together

Remember: Encapsulation hides data and bundles it with methods.

Next

Easy

5.In c++, which access modifier makes a class member visible everywhere?
a. private
b. public
c. protected
d. internal

Hint

Answer: b. Public

Remember: public allows unrestricted access across the program.

Next

Good job! 5/5 ★★★★★

You obtain a badge:

Concept Explorer

Starter Challenger

Next

4/5 ★★★★

You obtain a badge:

Starter Challenger

Next

3/5 ★★★

You obtain a badge:

Starter Challenger

Next

2/5 ★★

Don’t worry — review Classes & Objects and try again!

Next

1/5 ★

Don’t worry — review Classes & Objects and try again!

Next

0/5

Don’t worry — review Classes & Objects and try again!

Next

Pre test Moderate

- This pre-test consists of 5 multiple-choice questions on OOP concepts and basic applications.
- Read each question carefully and select the best answer.
- You have 8 minutes to finish.
- You start with 5 lives (hearts); each wrong answer deducts one life. Losing all lives ends the test.
- Hints are available, but using one deducts 1 point from your score.
- Your badge will be based on your performance.
- Completing the pre-test is required.

Next

Are you ready?

Proceed

Moderate

8:00

1.Given the code snippet below, what will be the output?

```
class Animal {
  public virtual void Sound() {
    Console.WriteLine("Animal sound");
  }
}
class Dog : Animal {
  public override void Sound() {
    Console.WriteLine("Bark");
  }
}
class Program {
  static void Main(string[ ] args) {
    Animal myDog = new Dog();
    myDog.Sound();
  }
}
```

Hint

Answer: b. bark

Remember: Polymorphism allows myDog.Sound() to use the Dog implementation.

Next

Moderate

8:00

a. animal sound
b. bark
c. error in compilation
d. no output

2.Which access modifier allows a class member to be accessible only within its class and derived classes?
a. private
b. protected
c. public
d. internal

Hint

Answer: b. protected

Remember: protected allows access within the same class and subclasses.

Next

Moderate

8:00

3.What will happen if you create an object of an abstract class in C++?
a. It will compile successfully.
b. It will cause a runtime error.
c. It will cause a compilation error.
d. It will create an empty object.

Hint

Answer: c. It will cause a compilation error

Remember: Abstract classes cannot be instantiated.

4.In C++, which of the following best demonstrates encapsulation?
a. Declaring a variable as public
b. Grouping methods and variables inside a class and making them private
c. Overriding a method in a child class
d. Writing methods with the same name but different parameters

Hint

Answer: b. Grouping methods and variables inside a class and making them private

Remember: Encapsulation hides implementation and protects data.

Next

Moderate

8:00

5.Suppose you created a generic class Box<T>. What does <T> represent?

a. A type parameter that can be replaced with any data type
b. A constant value for a class
c. A loop variable
d. A default C++ type

Hint

Answer: a. A type parameter that can be replaced with any data type

Remember: Generics allow defining a class with placeholders for data types.

Next

Good job! 5/5 ★★★★★

You obtain a badge:

Concept Challenger

Resilient Learner

Level-Up Learner

Next

4/5 ★★★★

You obtain a badge:

In Progress

Next

3/5 ★★★

You obtain a badge:

Level-Up Learner

Next

2/5 ★★

Don’t give up, every challenge is a step toward growth.
Review and try again!

Next

1/5 ★

Don’t give up, every challenge is a step toward growth.
Review and try again!

Next

0/5

Don’t give up, every challenge is a step toward growth.
Review and try again!

Next

Pre test Challenging

- This pre-test consists of 5 scenario-based questions that require critical thinking and debugging.
- You have 5 minutes to complete the test, so manage your time wisely.
- You start with 3 lives; each wrong answer deducts one life.
- Only 1 hint is available for the entire test, and using it deducts 1 point from your score.
- Your score and badge will be based on performance.
- Completing the pre-test is required.

Next

Are you ready?

Proceed

Challenging

5:00

1.Debuggine scenario: What will be the output of the following code?

```
class A {
  public virtual void Show() {
    Console.WriteLine("Class A");
  }
}
class B : A {
  public override void Show() {
    Console.WriteLine("Class B");
  }
}

}
```

Hint

Answer: b. Class B

Remember: new hides method in C, but since obj is type A, it calls the overridden method in B.

Next

Challenging

5:00

```
class C : B {
  public new void Show() {
    Console.WriteLine("Class C");
  }
}
class Program {
  static void Main() {
    A obj = new C();
    obj.Show();
  }
```

a. Class A
b. Class B
c. Class C
d. Error

Next

Challenging

5:00

2.Which of the following situations BEST demonstrates polymorphism in C++?
a. Having multiple classes share a single base class.
b. A subclass overriding a parent class method with its own implementation.
c. Creating multiple objects of the same class.
d. Declaring private variables inside a class.

Hint

Answer: b. A subclass overriding a parent class method with its own implementation.

Remember: This is method overriding, runtime polymorphism.

3.Consider this code snippet:

```
List<int> numbers = new List<int>(){1,2,3,4,5};
var result = numbers.Where(n => n % 2 == 0).ToList();
Console.WriteLine(result[0]);
```

Hint

Answer: b. 2

Remember: LINQ filters even numbers, [2,4]. First element is 2.

Next

Challenging

5:00

What will be printed?
a. 1
b. 2
c. 4
d. Error

4.Which of the following is the MOST accurate statement about interfaces in C++?
a. Interfaces can contain both implementation and declarations.
b. A class can inherit only one interface.
c. Interfaces support multiple inheritance of type definitions.
d. Interfaces are slower than classes and not recommended.

Hint

Answer: c. Interfaces support multiple inheritance of type definitions.

Remember: A class can implement multiple interfaces in C#.

Next

Challenging

5:00

5.You are tasked to design a program using encapsulation. Which of the following best applies it?
a. Creating private fields and exposing them through public properties.
b. Writing a method with the same name in different classes.
c. Creating multiple constructors with different parameters.
d. Using an abstract method inside an abstract class.

Hint

Answer: a. Creating private fields and exposing them through public properties.

Remember: Encapsulation = data hiding + controlled access.

Next

Good job! 5/5 ★★★★★

You obtain a badge:

Master Challenger

Persistent Learner

On the Climb

Next

4/5 ★★★★

You obtain a badge:

Persistent Learner

Next

3/5 ★★★

You obtain a badge:

On the Climb

Next

2/5 ★★

Don’t stop here — every failure is a step toward mastery. Try again!

Next

1/5 ★

Don’t stop here — every failure is a step toward mastery. Try again!

Next

0/5

Don’t stop here — every failure is a step toward mastery. Try again!

Next

Pre test Tough

- This pre-test consists of 5 advanced problem-solving questions.
- You have 4 minutes to finish, with a strict timer enforced.
- You start with 2 lives; each wrong answer deducts one life. Losing all ends the test immediately.
- No hints are available—you must rely on your knowledge and persistence.
- Only the highest badges are awarded at this level.
- Completing the pre-test is required.

Next

Are you ready?

Proceed

Tough

4:00

1.What will be the output of this program?

```
interface IShape {
  double Area();
}
class Circle : IShape {
  public double Radius { get; set; }
  public Circle(double r) { Radius = r; }
  public double Area() { return Math.PI * Radius * Radius; }
}
class Program {
  static void Main() {
    IShape s = new Circle(2);
    Console.WriteLine(s.Area().ToString(\"F2\"));
  }
}
```

Answer: b. 12.57

Remember: Area = $\pi * r^2 = 3.14 * 4 \approx 12.57$

Next

Tough

4:00

a. 6.28
b. 12.57
c. 3.14
d. Error

2.Which of the following BEST demonstrates exception handling in C++?
a. Using if-else for error checking.
b. Wrapping risky code inside a try block with a catch to handle errors.
c. Declaring variables as private for safety.
d. Writing a program without bugs.

Answer: b. Wrapping risky code inside a try block with a catch to handle errors.

Remember: Exception handling ensures program flow despite runtime errors.

Next

Tough

4:00

3.Consider the code:

```
class Base {
  public virtual void Show() { Console.WriteLine("Base"); }
}
class Derived : Base {
  public override void Show() { Console.WriteLine("Derived");
}
}
class Program {
  static void Main() {
    Base obj = new Derived();
    obj.Show();
  }
}
```

Answer: b. Derived

Remember: Virtual + override allows runtime polymorphism.

Next

Tough

4:00

What is the output?
a. base
b. Derived
c. Error
d. Both Base and Derived

4.Which LINQ query returns all odd numbers from the list numbers?
a. var result = numbers.Where(n => n % 2 == 0);
b. var result = numbers.Select(n => n % 2 != 0);
c. var result = numbers.Where(n => n % 2 != 0);
d. var result = numbers.Find(n => n % 2 != 0);

Answer: c. numbers.Where(n => n % 2 != 0)

Remember: Where filters items by condition; != 0 means odd numbers.

Next

Tough

4:00

5.Which scenario shows the use of multithreading in C++?
a. Saving data to a file using one process.
b. Running multiple tasks (e.g., downloading and processing) at the same time.
c. Calling methods in a sequence.
d. Writing code in multiple classes.

Answer: b. Running multiple tasks (e.g., downloading and processing) at the same time.

Remember: Multithreading allows concurrent execution.

Next

Good job! 5/5 ★★★★★

You obtain a badge:

OOP Master

Persistent Challenger

Code Warrior

Advancing

Next

4/5 ★★★★

You obtain a badge:

Code Warrior

Persistent Challenger

Next

3/5 ★★★

You obtain a badge:

Advancing

Next

2/5 ★★

This is the toughest stage — persistence leads to mastery. Review and try again

Next

1/5 ★

This is the toughest stage — persistence leads to mastery. Review and try again

Next

0/5

Don’t stop here — every failure is a step toward mastery. Try again!

Next

My Learning

Encapsulation Abstraction Inheritance Polymorphism

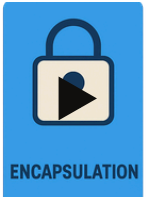
Start a weekly streak
Let's chip away at your learning goals.

0 weeks
Current Streak

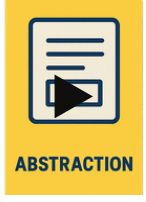
0/0 course min
2/1 visit
September 19 - 20

Inheritance
0% complete

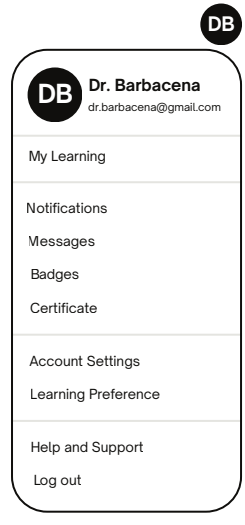
Polymorphism
0% complete



Encapsulation
0% complete



Abstraction
0% complete


DB
DB
Dr. Barbacena
dr.barbacena@gmail.com
My Learning
Notifications
Messages
Badges
Certificate
Account Settings
Learning Preference
Help and Support
Log out

Dr. Barbacena Easy Level Badges

Complete all badges to get your certificate!

Concept Explorer

Starter Challenger

Junior Inheritor

Code reuser

Data Protector

Guardian of Code

Essential Thinker

Rapid Coder

Polymorph Pioneer

Code Apprentice

OOP Explorer

Foundation Builder

Beginner Explorer

Abstract Thinker

Dr. Barbacena Moderate Level Badges

Complete all badges to get your certificate!

Concept Challenger

Resilient Learner

Level-Up Learner

Bridge builder

Data Guardian

Visionary Designer

Shape shifter

OOP Challenger

Code Warrior

Problem Solver

Dr. Barbacena Challenging Level Badges

Complete all badges to get your certificate!

Master Challenger

Persistent Learner

On the Climb

Syntax Slayer

Logic Breaker

Bug Hunter

Code Survivor

Code Gladiator

Persistent Debugger

Problem Solver

Dr. Barbacena Tough Level Badges

Complete all badges to get your certificate!

OOP Master

Persistent Challenger

Code Warrior

Advancing

Code Conqueror

Guardian of Syntax

Logic Seeker

Summit Climber

Grandmaster of code

Iron Coder

Unbreakable Spirit

Notifications

No notifications.

Messages

You have 0 unread messages.

Compose Unread

Search

You have no messages.

Select a message thread to read it here.

DB

Dr. Barbacena

View Public Profile

Profile

Learning Preference

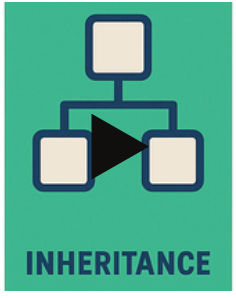
Account Security

Log out

LEARNER

Dr. Barbacena

Learning



Inheritance

0% complete

DB

Dr. Barbacena

View Public Profile

Profile

Learning Preference

Account Security

Log out

DB

Dr. Barbacena

View Public Profile
Profile
Learning Preference
Account Security
Log out

Profile

Add information about yourself

Basics

Write first name

Write last name

Occupation

Write your occupation

Biography

Write your biography

Save

DB

Dr. Barbacena

View Public Profile

Profile

Learning Preference

Account Security

Log out

DB

Dr. Barbacena

View Public Profile
Profile
Learning Preference
Account Security
Log out

Learning Preference

Dark mode/Light mode

Dark mode

Enable / disable motivational prompts

Enable

Pace setting

Daily reminders

Weekly reminders

DB

Dr. Barbacena

View Public Profile

Profile

Learning Preference

Account Security

Log out

DB

Dr. Barbacena

View Public Profile
Profile
Learning Preference
Account Security
Log out

Account Security

Edit your account settings and change your password here.

Email

Write your email

New Password

Enter new password

Confirm new password

Re-type new password

Save

DB

Dr. Barbacena

View Public Profile

Profile

Learning Preference

Account Security

Log out

You've successfully logged out of eMódulo. Come back soon!

Overview

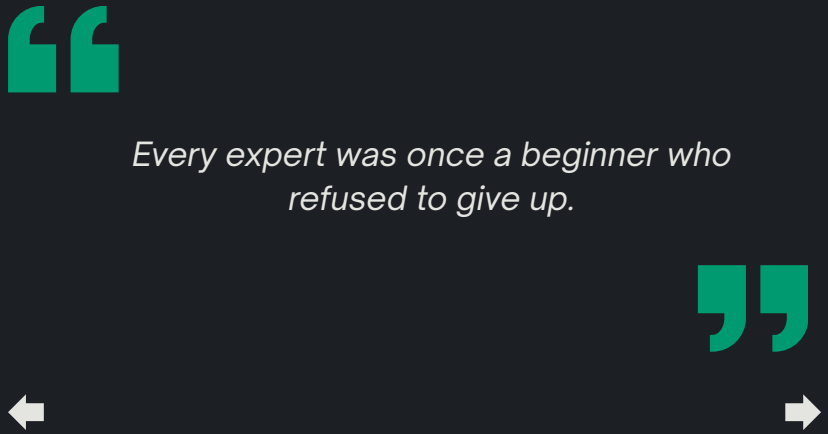
Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 1: Course Overview (easy level)

1. Course introduction
   7 mins.
2. Motivational Quote
   1 min.
3. Course Objectives
   3 mins.

Section 2: Inheritance

4. What is Inheritance?
   2 mins.
5. Why is Inheritance important?
   2 mins.
6. C++ Example
   3 mins.



Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 1: Course Overview (easy level)

- 1. Course introduction — 7 mins.
- 2. Motivational Quote — 1 min.
- 3. Course Objectives — 3 mins.

Section 2: Inheritance

- 4. What is Inheritance? — 2 mins.
- 5. Why is Inheritance important? — 2 mins.
- 6. C++ Example — 3 mins.

Course Objectives:

- **CO1.** Define the four OOP concepts--Abstraction, Encapsulation, Inheritance, and Polymorphism.
- **CO2.** Apply the four OOP concepts by writing basic C++ class structures, member functions, and simple program implementations.
- **CO3.** Analyze C++ code to identify correct and incorrect uses of Abstraction, Encapsulation, Inheritance, and Polymorphism.
- **CO4.** Evaluate different C++ program designs by comparing how effectively they implement the four OOP concepts.
- **CO5.** Create a functional C++ program that integrates all four OOP concepts to solve a defined programming task.

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 1: Course Overview (easy level)

1. Course introduction — 7 mins.
2. Motivational Quote — 2 mins.
3. Course Objectives — 3 mins.

Section 2: Inheritance

4. What is Inheritance? — 2 mins.
5. Why is Inheritance important? — 2 mins.
6. C++ Example — 3 mins.

What is Inheritance?

- In object-oriented programming (OOP), inheritance is a process of one class (derived class) inheriting properties and behaviors of an existing class (base class). This enhances the concept of reusing code through the inheritance of the shared attributes and functions in derived classes instead of copying them. The derived class can then inherit or alter the inherited characteristics giving rise to a specialized version of the base class.

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 1: Course Overview (easy level)

1. Course introduction — 7 mins.
2. Motivational Quote — 2 mins.
3. Course Objectives — 3 mins.

Section 2: Inheritance

4. What is Inheritance? — 2 mins.
5. Why is Inheritance important? — 2 mins.
6. C++ Example — 3 mins.

Why is Inheritance Important?

- Promotes code reusability, no need to repeat code.
- Makes programs organized and maintainable.
- Establishes hierarchy between general and specific classes.

Real-world example:

- Base Class: Vehicle → can start and stop.
- Derived Class: Car → inherits start/stop and adds driving functionality.

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 1: Course Overview (easy level)

1. Course introduction — 7 mins.
2. Motivational Quote — 2 mins.
3. Course Objectives — 3 mins.

Section 2: Inheritance

4. What is Inheritance? — 2 mins.
5. Why is important and real world example — 2 mins.
6. C++ Example — 3 mins.

C++ code example:

```
#include <iostream>
using namespace std;

class Vehicle {
public:
    void start() {
        cout << "Vehicle starting..." << endl;
    }
};
```

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 1: Course Overview (easy level)

- 1. Course introduction — 7 mins.
- 2. Motivational Quote — 2 mins.
- 3. Course Objectives — 3 mins.

Section 2: Inheritance

- 4. What is Inheritance? — 2 mins.
- 5. Why is important and real world example — 2 mins.
- 6. C++ Example — 3 mins.

```
class Car : public Vehicle {  // Inherits from Vehicle
public:
    void drive() {
        cout << "Car is driving..." << endl;
    }
};

int main() {
    Car myCar;
    myCar.start();  // Inherited
    myCar.drive();  // Car-specific
    return 0;
}
```

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 1: Course Overview (easy level)

- 1. Course introduction — 7 mins.
- 2. Motivational Quote — 2 mins.
- 3. Course Objectives — 3 mins.

Section 2: Inheritance

- 4. What is Inheritance? — 2 mins.
- 5. Why is important and real world example — 2 mins.
- 6. C++ Example — 5 mins.

Output:

- Vehicle starting...
- Car is driving...

Key points to remember:

- Use : to show inheritance (class Car : public Vehicle).
- Derived classes reuse methods of base classes.
- Derived classes can add their own features.

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 2: Inheritance

- 7. Output & Key points to remember — 3 mins.
- 8. Activity — 10 mins.
- 9. Completion of Inheritance — 1 min.

Section 4: Encapsulation

- 10. Motivational Quote — 2 mins.
- 11. What is Encapsulation and why it is Important? — 2 mins.
- 12. C++ Example — 5 mins.
- 13. Activity — 10 mins.

Activity Easy

- Drag the correct syntax blocks into the blanks to complete the inheritance program.
- Hints are available (no penalty on Easy level).
- No timer, take your time to understand each question.
- You must answer correctly to finish the activity.
- Complete the activity to earn a badge.

Proceed to Activity

Activity

```
#include <iostream>
using namespace std;

class Vehicle {
public:
    void start() {
        cout << "Vehicle starting..." << endl;
    }
};
```

Hint

Hint:

“Which function was defined inside Vehicle to make the car start?”

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 2: Inheritance

- 7. Output & Key points to remember — 3 mins.
- 8. Activity — 10 mins.
- 9. Completion of Inheritance — 1 min.

Section 4: Encapsulation

- 10. Motivational Quote — 2 mins.
- 11. What is Encapsulation and why it is Important? — 2 mins.
- 12. C++ Example — 5 mins.
- 13. Activity — 10 mins.

```
class Car _____ Vehicle {   // (1) Fill in the blank
public:
    void drive() {
        cout << "Car is driving..." << endl;
    }
};

int main() {
    Car myCar;
    myCar._____();   // (2) Fill in the blank
    myCar.drive();
    return 0;
}
```

: public | : private | start | drive

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 2: Inheritance

- 7. Output & Key points to remember — 3 mins.
- 8. Activity — 10 mins.
- 9. Completion of Inheritance — 1 min.

Section 4: Encapsulation

- 10. Motivational Quote — 2 mins.
- 11. What is Encapsulation and why it is Important? — 2 mins.
- 12. C++ Example — 5 mins.
- 13. Activity — 10 mins.

5/5

GOOD JOB!

New badge unlock!

Junior Inheritor

Overview

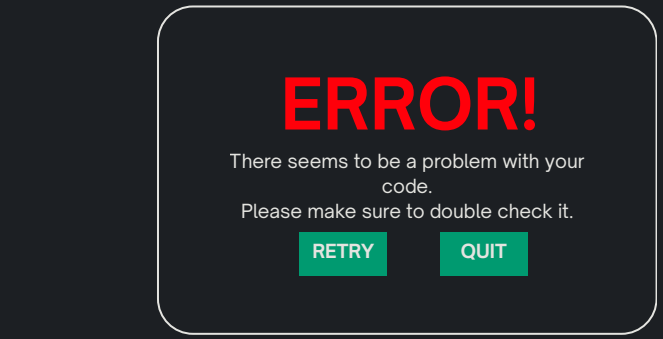
Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 2: Inheritance

- 7. Output & Key points to remember
  - 3 mins.
- 8. Activity
  - 10 mins.
- 9. Completion of Inheritance
  - 1 min.

Section 4: Encapsulation

- 10. Motivational Quote
  - 2 mins.
- 11. What is Encapsulation and why it is Important?
  - 2 mins.
- 12. C++ Example
  - 5 mins.
- 13. Activity
  - 10 mins.



Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 2: Inheritance

- 7. Output & Key points to remember — 3 mins.
- 8. Activity — 10 mins.
- 9. Completion of Inheritance — 1 min.

Section 4: Encapsulation

- 10. Motivational Quote — 2 mins.
- 11. What is Encapsulation and why it is Important? — 2 mins.
- 12. C++ Example — 5 mins.
- 13. Activity — 10 mins.

Congratulations on completing Inheritance!

New badge unlock!

Code reuser

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 2: Inheritance

- 7. Output & Key points to remember — 3 mins.
- 8. Activity — 10 mins.
- 9. Completion of Inheritance — 1 min.

Section 4: Encapsulation

- 10. Motivational Quote — 2 mins.
- 11. What is Encapsulation and why it is Important? — 2 mins.
- 12. C++ Example — 5 mins.
- 13. Activity — 10 mins.

The secret of getting ahead is getting started

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 3: Inheritance

- 7. Output & Key points to remember
  3 mins.
- 8. Activity
  10 mins.
- 9. Completion of Inheritance
  1 min.

Section 4: Encapsulation

- 10. Motivational Quote
  2 mins.
- 11. What is Encapsulation and why it is Important?
  2 mins.
- 12. C++ Example
  5 mins.
- 13. Activity
  10 mins.

What is Encapsulation?

- Encapsulation in OOP is the process of bundling data (attributes) and the methods (functions) that operate on that data into a single unit, typically a class. It involves hiding the internal state of an object and controlling access to it through public methods (like getters and setters), thereby protecting the data from unauthorized modification and ensuring data integrity.

Why is Encapsulation important?

- Provides data protection.
- Prevents accidental changes to variables.
- Makes programs more secure and reliable.

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 3: Inheritance

- 7. Output & Key points to remember
  3 mins.
- 8. Activity
  10 mins.
- 9. Completion of Inheritance
  1 min.

Section 4: Encapsulation

- 10. Motivational Quote
  2 mins.
- 11. What is Encapsulation and why it is Important?
  2 mins.
- 12. C++ Example
  5 mins.
- 13. Activity
  10 mins.

C++ code example:

```
#include <iostream>
using namespace std;

class Account {
private:
   double balance;   // hidden data

public:
   // setter method
   void setBalance(double b) {
      if (b >= 0) balance = b;  // validation
   }
```

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 3: Inheritance

- 7. Output & Key points to remember — 3 mins.
- 8. Activity — 10 mins.
- 9. Completion of Inheritance — 1 min.

Section 4: Encapsulation

- 10. Motivational Quote — 2 mins.
- 11. What is Encapsulation and why it is Important? — 2 mins.
- 12. C++ Example — 5 mins.
- 13. Activity — 10 mins.

C++ code example:

```
// getter method
    double getBalance() {
        return balance;
    }
};

int main() {
    Account myAcc;
    myAcc.setBalance(5000);   // access via method
    cout << "Balance: " << myAcc.getBalance() << endl;
    return 0;
}
```

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 3: Inheritance

- 7. Output & Key points to remember — 3 mins.
- 8. Activity — 10 mins.
- 9. Completion of Inheritance — 1 min.

Section 4: Encapsulation

- 10. Motivational Quote — 2 mins.
- 11. What is Encapsulation and why it is Important? — 2 mins.
- 12. C++ Example — 5 mins.
- 13. Activity — 10 mins.

Output:

- Balance: 5000

Key points to remember:

- private hides variables inside the class.
- Use getter and setter methods to control access.
- Encapsulation protects data and ensures safe interaction with objects.

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 3: Inheritance

- 7. Output & Key points to remember
  3 mins.
- 8. Activity
  10 mins.
- 9. Completion of Inheritance
  1 min.

Section 4: Encapsulation

- 10. Motivational Quote
  2 mins.
- 11. What is Encapsulation and why it is Important?
  2 mins.
- 12. C++ Example
  5 mins.
- 13. Activity
  10 mins.

Activity Easy

- Drag the correct syntax blocks into the blanks to complete the encapsulation code.
- Hints are available (no penalty on Easy level).
- No timer, take your time to understand each question.
- Answer correctly to complete the activity and earn a badge.

Proceed to Actvity

Activity

```
class Student {
private:
   int age;   // (1) _____ data

public:
   void setAge(int a) { age = a; }   // setter
   int getAge() { return age; }      // (2) _____
};
```

Hint

Hint:

"If setAge() is a setter, what do we call the method that retrieves the value?"

private | protected | hidden | getter

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 3: Inheritance

- 7. Output & Key points to remember — 3 mins.
- 8. Activity — 10 mins.
- 9. Completion of Inheritance — 1 min.

Section 4: Encapsulation

- 10. Motivational Quote — 2 mins.
- 11. What is Encapsulation and why it is Important? — 2 mins.
- 12. C++ Example — 5 mins.
- 13. Activity — 10 mins.

5/5

GOOD JOB!

New badge unlock!

Data Protector

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 3: Inheritance

- 7. Output & Key points to remember
  - 3 mins.
- 8. Activity
  - 10 mins.
- 9. Completion of Inheritance
  - 1 min.

Section 4: Encapsulation

- 10. Motivational Quote
  - 2 mins.
- 11. What is Encapsulation and why it is Important?
  - 2 mins.
- 12. C++ Example
  - 5 mins.
- 13. Activity
  - 10 mins.

Congratulations on completing Encapsulation!

New badge unlock!

Guardian of Code

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 4: Encapsulation

- 14. Completion of Encapsulation
  - 1 min.

Section 5: Abstraction

- 15. Motivational Quote
  - 2 mins.
- 16. What is abstraction?
  - 2 mins.
- 17. Why is Abstraction Important?
  - 2 mins.
- 18. C++ Example
  - 5 mins.
- 19. Activity
  - 10 mins.
- 20. Completion of Abstraction
  - 1 min.

Focus on what matters; hide what doesn't

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 4: Inheritance

- 14. Completion of Encapsulation
  1 min.

Section 5: Abstraction

- 15. Motivational Quote
  2 mins.
- 16. What is abstraction?
  2 mins.
- 17. Why is Abstraction Important?
  2 mins.
- 18. C++ Example
  5 mins.
- 19. Activity
  10 mins.
- 20. Completion of Abstraction
  1 min.

What is Abstraction?

- In object-oriented programming (OOP), abstraction is the process of hiding complex internal implementation details and displaying only the essential, relevant features of an object to the user or other programs. It simplifies complex systems by showing only the important functionality, reducing complexity and increasing efficiency.

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 4: Inheritance

- 14. Completion of Encapsulation
  1 min.

Section 5: Abstraction

- 15. Motivational Quote
  2 mins.
- 16. What is abstraction?
  2 mins.
- 17. Why is Abstraction Important?
  2 mins.
- 18. C++ Example
  5 mins.
- 19. Activity
  10 mins.
- 20. Completion of Abstraction
  1 min.

Why is Abstraction Important??

- Reduces complexity by hiding details. (CO4)
- Focuses only on what is needed. (CO6)
- Makes code easier to maintain and extend. (CO8)

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 4: Inheritance

- 14. Completion of Encapsulation
  1 min.

Section 5: Abstraction

- 15. Motivational Quote
  2 mins.
- 16. What is abstraction?
  2 mins.
- 17. Why is Abstraction Important?
  2 mins.
- 18. C++ Example
  5 mins.
- 19. Activity
  10 mins.
- 20. Completion of Abstraction
  1 min.

C++ code example:

```
#include <iostream>
using namespace std;

// Abstract class
class Shape {
public:
    virtual void draw() = 0;  // Pure virtual function
};
```

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 4: Inheritance

- 14. Completion of Encapsulation
  - 1 min.

Section 5: Abstraction

- 15. Motivational Quote
  - 2 mins.
- 16. What is abstraction?
  - 2 mins.
- 17. Why is Abstraction Important?
  - 2 mins.
- 18. C++ Example
  - 5 mins.
- 19. Activity
  - 10 mins.
- 20. Completion of Abstraction
  - 1 min.

C++ code example:

```
class Circle : public Shape {
public:
    void draw() {
        cout << "Drawing a Circle" << endl;
    }
};

int main() {
    Shape* s = new Circle();
    s->draw();   // Calls Circle's implementation
    delete s;
    return 0;
}
```

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 4: Inheritance

- 14. Completion of Encapsulation
  1 min.

Section 5: Abstraction

- 15. Motivational Quote
  2 mins.
- 16. What is abstraction?
  2 mins.
- 17. Why is Abstraction Important?
  2 mins.
- 18. C++ Example
  5 mins.
- 19. Activity
  10 mins.
- 20. Completion of Abstraction
  1 min.

Output:

Drawing a Circle

Key points to remember:

- Abstraction in C++ is achieved using abstract classes and pure virtual functions.
- Declaring a function with = 0 makes it pure virtual (no implementation in base class).
- Derived classes must implement the abstract method.

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 4: Inheritance

- 14. Completion of Encapsulation
  1 min.

Section 5: Abstraction

- 15. Motivational Quote
  2 mins.
- 16. What is abstraction?
  2 mins.
- 17. Why is Abstraction Important?
  2 mins.
- 18. C++ Example
  5 mins.
- 19. Activity
  10 mins.
- 20. Completion of Abstraction
  1 min.

Activity Easy

- Drag the correct syntax blocks into the blanks to complete the abstraction example.
- Hints are available (no penalty on Easy level).
- No timer, take your time to understand the questions.
- Answer correctly to complete the activity and earn a badge.

Proceed to Actvity

Activity:

```
class Animal {
public:
    virtual void makeSound() = _____ ;  // (1) Fill in
};

class Dog : public Animal {
public:
    void makeSound() {
        cout << "Barking..." << endl;
    }
};
```

Hint

Hint:

"In C++, a pure virtual function is declared by assigning it the value _____. This makes the class abstract and forces derived classes to implement it."

0 | ; | null | : public

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 4: Inheritance

- 14. Completion of Encapsulation
  - 1 min.

Section 5: Abstraction

- 15. Motivational Quote
  - 2 mins.
- 16. What is abstraction?
  - 2 mins.
- 17. Why is Abstraction Important?
  - 2 mins.
- 18. C++ Example
  - 5 mins.
- 19. Activity
  - 10 mins.
- 20. Completion of Abstraction
  - 1 min.

5/5

GOOD JOB!

New badge unlock!

Essential Thinker

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 4: Inheritance

- 14. Completion of Encapsulation
  - 1 min.

Section 5: Abstraction

- 15. Motivational Quote
  - 2 mins.
- 16. What is abstraction?
  - 2 mins.
- 17. Why is Abstraction Important?
  - 2 mins.
- 18. C++ Example
  - 5 mins.
- 19. Activity
  - 10 mins.
- 20. Completion of Abstraction
  - 1 min.

Remember

1.In C++, a pure virtual function is declared by assigning = 0 at the end of the function prototype:
virtual void makeSound() = 0;

2. This tells the compiler:

- The function has no implementation in the base class (Animal).
- The class containing it (Animal) becomes an abstract class.

3. Any class derived from Animal (like Dog) must override/implement the pure virtual function.

```
void makeSound() {
   cout << "Barking..." << endl;
}
```

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 4: Inheritance

- 14. Completion of Encapsulation
  1 min.

Section 5: Abstraction

- 15. Motivational Quote
  2 mins.
- 16. What is abstraction?
  2 mins.
- 17. Why is Abstraction Important?
  2 mins.
- 18. C++ Example
  5 mins.
- 19. Activity
  10 mins.
- 20. Completion of Abstraction
  1 min.

Remember

4. Without = 0, Animal would not be abstract, and the compiler would expect a definition for makeSound().

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 4: Inheritance

- 14. Completion of Encapsulation
  1 min.

Section 5: Abstraction

- 15. Motivational Quote
  2 mins.
- 16. What is abstraction?
  2 mins.
- 17. Why is Abstraction Important?
  2 mins.
- 18. C++ Example
  5 mins.
- 19. Activity
  10 mins.
- 20. Completion of Abstraction
  1 min.

Congratulations on completing Abstraction!

New badge unlock!

Abstract Thinker

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 4: Inheritance

- 14. Completion of Encapsulation
  - 1 min.

Section 5: Abstraction

- 15. Motivational Quote
  - 2 mins.
- 16. What is abstraction?
  - 2 mins.
- 17. Why is Abstraction Important?
  - 2 mins.
- 18. C++ Example
  - 5 mins.
- 19. Activity
  - 10 mins.
- 20. Completion of Abstraction
  - 1 min.

Flexibility is the key to stability.

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 6: Polymorphism

- 21. Motivational Quote
  1 min.
- 22. What is polymorphism?
  2 mins.
- 23. Types of polymorphism?
  2 mins.
- 24. C++ Example?
  2 mins.
- 25. Activity
  10 mins.
- 26. Completion of Polymorphism
  1 min.
- 27. Post-test
  10mins.

What is Polymorphism?

- Polymorphism in Object-Oriented Programming (OOP) is the ability of objects to take on multiple forms or behaviors, allowing different objects to be treated as instances of a common parent class while retaining their unique implementations. This core OOP concept enables code to be more flexible, reusable, and maintainable by providing a single interface to diverse underlying classes, making programs adaptable to new situations and easier to manage.

Section 6: Polymorphism

- 21. Motivational Quote — 1 min.
- 22. What is polymorphism? — 2 mins.
- 23. Types of polymorphism? — 2 mins.
- 24. C++ Example? — 2 mins.
- 25. Activity — 10 mins.
- 26. Completion of Polymorphism — 1 min.
- 27. Post-test — 10mins.

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Types of polymorphism

- Compile-time (Static) → Function overloading, operator overloading.
- Run-time (Dynamic) → Achieved using virtual functions.

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 6: Polymorphism

- 21. Motivational Quote — 1 min.
- 22. What is polymorphism? — 2 mins.
- 23. Types of polymorphism? — 2 mins.
- 24. C++ Example — 2 mins.
- 25. Activity — 10 mins.
- 26. Completion of Polymorphism — 1 min.
- 27. Post-test — 10mins.

C++ code example:

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void makeSound() {
        cout << "Some generic animal sound" << endl;
    }
};
```

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 6: Polymorphism

- 21. Motivational Quote — 1 min.
- 22. What is polymorphism? — 2 mins.
- 23. Types of polymorphism? — 2 mins.
- 24. C++ Example — 2 mins.
- 25. Activity — 10 mins.
- 26. Completion of Polymorphism — 1 min.
- 27. Post-test — 10mins.

C++ code example:

```
class Dog : public Animal {
public:
    void makeSound() override {
        cout << "Barking..." << endl;
    }
};

class Cat : public Animal {
public:
    void makeSound() override {
        cout << "Meowing..." << endl;
    }
};
```

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 6: Polymorphism

- 21. Motivational Quote — 1 min.
- 22. What is polymorphism? — 2 mins.
- 23. Types of polymorphism? — 2 mins.
- 24. C++ Example — 2 mins.
- 25. Activity — 10 mins.
- 26. Completion of Polymorphism — 1 min.
- 27. Post-test — 10mins.

C++ code example:

```
int main() {
    Animal* a1 = new Dog();
    Animal* a2 = new Cat();

    a1->makeSound();  // Calls Dog's version
    a2->makeSound();  // Calls Cat's version

    delete a1;
    delete a2;
    return 0;
}
```

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 6: Polymorphism

- 21. Motivational Quote — 1 min.
- 22. What is polymorphism? — 2 mins.
- 23. Types of polymorphism? — 2 mins.
- 24. C++ Example — 2 mins.
- 25. Activity — 10 mins.
- 26. Completion of Polymorphism — 1 min.
- 27. Post-test — 10mins.

Output:

Barking...
Meowing...

Key points to remember:

- Polymorphism = one interface, multiple implementations.
- Achieved in C++ by virtual functions (dynamic binding).
- Increases flexibility and reusability of code.

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 6: Polymorphism

- 21. Motivational Quote — 1 min.
- 22. What is polymorphism? — 2 mins.
- 23. Types of polymorphism? — 2 mins.
- 24. C++ Example — 2 mins.
- 25. Activity — 10 mins.
- 26. Completion of Polymorphism — 1 min.
- 27. Post-test — 10mins.

Activity Easy

- Drag and drop the correct keyword to enable polymorphism.
- Hints are available (no penalty on Easy level).
- No timer, take your time to understand the questions.
- Complete the activity to earn a badge.
- All activities must be completed to progress.

Proceed to Actvity

Activity:

```
class Shape {
public:
    _____ void draw() {  // (1) Fill in
        cout << "Drawing a Shape" << endl;
    }
};
```

Hint

Hint:

"To enable runtime polymorphism, the base class method must be marked as _____. This tells C++ to look at the actual object type at runtime, not just the pointer type."

virtual | public | abstract | protected

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 6: Polymorphism

- 21. Motivational Quote — 1 min.
- 22. What is polymorphism? — 2 mins.
- 23. Types of polymorphism? — 2 mins.
- 24. C++ Example — 2 mins.
- 25. Activity — 10 mins.
- 26. Completion of Polymorphism — 1 min.
- 27. Post-test — 10mins.

5/5

GOOD JOB!

New badge unlock!

Rapid Coder

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 6: Polymorphism

- 21. Motivational Quote — 1 min.
- 22. What is polymorphism? — 2 mins.
- 23. Types of polymorphism? — 2 mins.
- 24. C++ Example — 2 mins.
- 25. Activity — 10 mins.
- 26. Completion of Polymorphism — 1 min.
- 27. Post-test — 10mins.

Remember:

1.In C++, the keyword virtual is used to achieve runtime polymorphism.
virtual void draw() { ... }

2. When a function is declared as virtual in a base class, it allows derived classes to override it with their own implementation.

3. At runtime, the program decides which version of the function to call based on the type of object being pointed to, this is called dynamic binding or late binding.

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 6: Polymorphism

- 21. Motivational Quote — 1 min.
- 22. What is polymorphism? — 2 mins.
- 23. Types of polymorphism? — 2 mins.
- 24. C++ Example — 2 mins.
- 25. Activity — 10 mins.
- 26. Completion of Polymorphism — 1 min.
- 27. Post-test — 10mins.

Congratulations on completing Polymorphism!

New badge unlock!

Polymorph Pioneer

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Section 6: Polymorphism

- 21. Motivational Quote — 1 min.
- 22. What is polymorphism? — 2 mins.
- 23. Types of polymorphism? — 2 mins.
- 24. C++ Example — 2 mins.
- 25. Activity — 10 mins.
- 26. Completion of Polymorphism — 1 min.
- 27. Post-test — 10mins.

Activity Moderate

- Each activity must be completed within 8 minutes.
- You have 5 lives; each wrong answer costs 1 life.
- Hints are available, but using one deducts 1 point.
- Complete the activity to earn a badge.
- All activities must be finished to progress.

Proceed to Actvity

Activity:

8:00

1. Complete the inheritance relationship.

Hint

Hint

"In C++, inheritance uses a colon + access specifier. Which base class method do we call to start the car?"

```
Class Vehicle {
public:
    void start() { cout << "Vehicle starting..." << endl; }
};

class Car _____ Vehicle {   // (1) Fill in
public:
    void drive() { cout << "Car is driving..." << endl; }
};
```

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Activity (Moderate level)

Inheritance

Activity:

8:00

```
int main() {
    Car myCar;
    myCar._____();   // (2) Fill in
    myCar.drive();
}
```

public | start | inherit | extends

Activity (Moderate level)

Inheritance

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

5/5

GOOD JOB!

New badge unlock!

Bridge builder

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Activity:

8:00

1. Protect the student's data using encapsulation.

Hint

```
class Student {
   _____:  // (1) Fill in
      int age;
public:
   void setAge(int a) { age = a; }
   int _____() { return age; }  // (2) Fill in
};
```

Hint

"Encapsulation hides data with private and provides methods like setter and _____."

getter | setter | private | public

Activity (Moderate level)

Encapsulation

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

5/5

GOOD JOB!

New badge unlock!

Data Guardian

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Activity:

8:00

1. Make this class abstract.

Hint

Hint

"In C++, a pure virtual function is declared by assigning it to ____."

```
class Shape {
public:
   virtual void draw() ____;  // (1) Fill in
};

class Circle : public Shape {
public:
   void draw() { cout << "Circle" << endl; }
};
```

= 0; virtual; abstract; { }

Activity (Moderate level)

Abstraction

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

5/5

GOOD JOB!

New badge unlock!

Visionary Designer

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Activity:

8:00

1. Enable runtime polymorphism for speak().

Hint

Hint

"In C++, a pure virtual function is declared by assigning it to ____."

```
class Animal {
public:
   _____ void speak() { cout << "Animal"; }  // (1) Fill in
};

class Dog : public Animal {
public:
   void speak() _____ { cout << "Bark"; }   // (2) Fill in
};
```

virtual | override | static | friend

Activity (Moderate level)

Polymorphism

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

5/5

GOOD JOB!

New badge unlock!

Shape shifter

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Activity Challenging

- You have 5 minutes to complete the pre-test, so manage your time wisely.
- You start with 3 lives; each wrong answer deducts one life.
- Only 1 hint is available for the entire test, and using it deducts 1 point.
- Scores and badges are awarded based on performance.
- Completing the pre-test is required.

Proceed to Actvity

Activity:

5:00

1. Fill in the missing keyword that allows class inheritance.

Hint

Hint

"In C++, inheritance uses colon (:) + access specifier."

```
class Person {
public:
    void walk() { cout << "Walking..."; }
};

class Student _____ Person {   // (1) Fill in
};
```

Activity (Challenging level)

Inheritance

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

5/5

GOOD JOB!

New badge unlock!

Syntax Slayer

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Activity:

5:00

1. Fill in the blanks to apply encapsulation in the Account class.

Hint

Hint

"Encapsulation = hide data with access specifier + use setters and getters."

```
class Account {
   _____:      // (1) Fill in
      double balance;

public:
   void setBalance(double b) { balance = b; }   // setter
   double _____() { return balance; }          // (2) Fill in
};
```

Activity (Challenging level)

Encapsulation

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

5/5

GOOD JOB!

New badge unlock!

Logic Breaker

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Activity:

5:00

1. Make Shape an abstract class by completing the pure virtual function.

Hint

```
class Shape {
public:
   virtual void draw() _____ ;  // (1) Fill in
};
```

Hint

"In C++, pure virtual functions are written with = 0."

Activity (Challenging level)

Abstraction

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

5/5

GOOD JOB!

New badge unlock!

Bug Hunter

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Activity:

5:00

1. Enable runtime polymorphism for the speak() function.

Hint

Hint

"Base uses one keyword to enable polymorphism, derived uses another to confirm overriding."

```
class Animal {
public:
    _____ void speak() { cout << "Generic Animal"; }  // (1) Fill
in
};

class Dog : public Animal {
public:
    void speak() _____ { cout << "Bark!"; }          // (2) Fill in
};
```

Activity (Challenging level)

Polymorphism

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

5/5

GOOD JOB!

New badge unlock!

Code Survivor

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Activity

Tough

- You have 4 minutes to complete the pre-test, so manage your time wisely.
- You start with 2 lives; each wrong answer deducts one life.
- Only 1 hint is available for the entire test, and using it deducts 1 point.
- Scores and badges are awarded based on performance.
- Completion is required to earn a certificate.

Proceed to Actvity

Activity:

4:00

1. Fix the missing inheritance and method call.

Hint

Hint

"Think of colon + access specifier for inheritance. Which method is inherited from Device?"

```
class Device {
public:
    void powerOn() { cout << "Powering on..."; }
};

class Phone _____ Device {   // (1) Fill in

public:
    void call() { cout << "Calling..."; }
};
```

Activity (Challenging level)

Inheritance

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Activity:

4:00

```
int main() {
    Phone myPhone;
    myPhone._____();   // (2) Fill in
    myPhone.call();
}
```

Activity (Challenging level)

Inheritance

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

5/5

GOOD JOB!

New badge unlock!

Code Conqueror

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Activity:

4:00

1. Secure the data with encapsulation.

```
class BankAccount {
   _____:    // (1) Fill in
      double balance;

public:
   void deposit(double amount) { balance += amount; }
   double _____() { return balance; }  // (2) Fill in
};
```

Hint

Hint

"Encapsulation hides variables with the strictest access level and uses a method to retrieve it."

Activity (Challenging level)

Encapsulation

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

5/5

GOOD JOB!

New badge unlock!

Guardian of Syntax

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Activity:

4:00

1. Force derived classes to implement their own method.

Hint

Hint

"Pure virtual functions in C++ are declared with = 0."

```
class Shape {
public:
    virtual void draw() _____ ;  // (1) Fill in
};

class Triangle : public Shape {
public:
    void draw() { cout << "Triangle"; }
};
```

Activity (Challenging level)

Abstraction

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

5/5

GOOD JOB!

New badge unlock!

Logic Seeker

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Activity:

4:00

1. Enable polymorphism and correct overriding.

Hint

Hint

"One keyword in base enables dynamic binding, another in derived confirms overriding."

```
class Animal {
public:
    _____ void speak() { cout << "Generic animal"; }   // (1) Fill in
};

class Cat : public Animal {
public:
    void speak() _____ { cout << "Meow"; }            // (2) Fill in
};
```

Activity (Challenging level)

Polymorphism

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

5/5

GOOD JOB!

New badge unlock!

Summit Climber

Overview

Learn the introduction and concepts of Object Oriented Programming. Complete course!

Post test Easy

- This post-test has 8 items (2 questions per OOP pillar).
- Read each question carefully and select the best answer.
- There is no time limit.
- You may use the hint button if needed (no penalty in Easy Level).
- At the end, you will see your score and earn a badge for completion.
- It is your requirement to finish the post-test.

Next

Are you ready?

Proceed

Easy

1. What does inheritance allow in Object-Oriented Programming?

a. One class to hide data from another
b. One class to acquire attributes and behaviors of another
c. One method to perform multiple tasks
d. One class to contain only abstract methods

Hint

Answer: b.

Remember: Inheritance allows a derived class to reuse properties and methods of a base class.

2.Which symbol is used in C++ to declare inheritance?

a. :
b. ;
c. { }
d. ( )

Hint

Answer: a.

Remember: class Child : public Parent uses : for inheritance.

Next

Easy

R

3.Which keyword in C++ is used to hide data inside a class?
a. public
b. private
c. protected
d. static

Hint

Answer: b.

Remember: private restricts access to data members.

U

4. Which method allows controlled access to private data?
a. getter and setter methods
b. loops and conditions
c. constructors
d. headers

Hint

Answer: a.

Remember: Encapsulation uses getter/setter methods to safely access private data.

Next

Easy

U

5. In C++, abstraction is implemented using:
a. Loops
b. Abstract classes and pure virtual functions
c. Templates
d. Constructors

Hint

Answer: b.

Remember: Abstraction hides implementation details using abstract classes and pure virtual functions.

R

6. What does declaring virtual void draw() = 0; mean?
a. The function is private
b. The function is overloaded
c. The function is a pure virtual function
d. The function is inline

Hint

Answer: c.

Remember: = 0 means pure virtual, no implementation in the base class.

Next

Easy

7. What does polymorphism mean in Object-Oriented Programming?

R

a. One object can inherit many classes
b. One function can have multiple forms
c. One class can contain only public methods
d. One function can only be static

Hint

Answer: b.

Remember: Polymorphism = “many forms”, function behaves differently depending on the object.

8. Which keyword in C++ enables runtime polymorphism?

R

a. abstract
b. protected
c. virtual
d. override

Hint

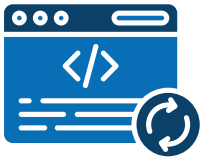
Answer: c.

Remember: Declaring a function as virtual allows derived classes to override it at runtime.

Next

Good job! 8/8

You obtain a badge:



Code Apprentice

OOP Explorer

Foundation Builder

Beginner Explorer

Next

Good job! 7/8

You obtain a badge:

OOP Explorer

Try again

Next

Good job! 6/8

You obtain a badge:

Foundation Builder

Try again

Next

Good job! 5/8

You obtain a badge:

Beginner Explorer

Try again

Next

3/8

This is the toughest stage — persistence leads to mastery. Review and try again

Try again

Next

2/8

This is the toughest stage — persistence leads to mastery. Review and try again

Try again

Next

1/8

This is the toughest stage — persistence leads to mastery. Review and try again

Try again

Next

0/8

This is the toughest stage — persistence leads to mastery. Review and try again

Try again

Next

Post test Moderate

- A total of 8 questions.
- Each must be answered within 8 minutes.
- You have 5 lives, a wrong answer deducts one life. If you lose all, the post-test ends.
- Hints available, but using one deducts 1 point from your score.
- Your badge depends on performance (higher score = stronger badge).
- Completing this test is a requirement for progression.

Next

Are you ready?

Proceed

Post test

Moderate

8:00

1. What will be the output:

```
class A {
public:
    void show() { cout << "Class A"; }
};
class B : public A {
public:
    void show() { cout << "Class B"; }
};
int main() {
    B obj;
    obj.show();
}
```

AP

Hint

Answer: b.

Remember: The show() method in B overrides A's version. Since obj is of type B, Class B is displayed.

Next

Post test

Moderate

8:00

a. Class A
b. Class B
c. Error
d. Both A and B

2. Fill in the blank to correctly inherit Car from Vehicle:

class Car _____ Vehicle { };

a. extends
b. : public
c. inherits
d. : protected

AP

Hint

Answer: b.

Remember: C++ uses : public to show inheritance. extends is for Java, not C++.

Next

Post test

Moderate

8:00

3. Why is encapsulation important in C++ classes?
a. To allow direct access to variables.
b. To hide data and control access using methods.
c. To avoid creating objects.
d. To make functions global.

AP

Hint

Answer: b.

Remember: Encapsulation hides sensitive data by using private members and controlling access via getters/setters.

Next

Post test

Moderate

8:00

4. What will the code output?

```
class Student {
private:
    int age;
public:
    void setAge(int a) { age = a; }
    int getAge() { return age; }
};

int main() {
    Student s;
    s.setAge(20);
    cout << s.getAge();
}
```

AP

Hint

Answer: b.

Remember: The setter updates age to 20, and the getter retrieves it, so the output is 20.

Next

Post test

Moderate

8:00

a. 0
b. 20
c. Error
d. Undefined

5. Which makes a class abstract in C++?
a. To allow direct access to variables.
b. To hide data and control access using methods.
c. To avoid creating objects.
d. To make functions global.

AP

Hint

Answer: a.

Remember: C++ does not use the abstract keyword. A class is abstract if it has at least one pure virtual function (= 0).

Next

Post test

Moderate

8:00

6. What happens if a derived class does not implement all pure virtual functions?
a. It becomes abstract itself
b. It compiles but errors at runtime
c. It implements them automatically
d. They are ignored

AN

Hint

Answer: a.

Remember: If a derived class does not override pure virtual functions, it remains abstract and cannot be instantiated.

Next

Post test

Moderate

8:00

7. What is the output?

```
class Animal {
public:
    virtual void speak() { cout << "Animal"; }
};
class Dog : public Animal {
public:
    void speak() override { cout << "Dog"; }
};
int main() {
    Animal* a = new Dog();
    a->speak();
}
```

AP

Hint

Answer: b.

Remember: Because of the virtual keyword, runtime polymorphism occurs. The pointer a calls Dog's overridden function, so the output is Dog.

Next

Post test

Moderate

8:00

a. Animal
b. Dog
c. AnimalDog
d. Error

8. Function overloading in C++ is resolved at:
a. Runtime
b. Compile-time
c. Link-time
d. Execution-time

AP

Hint

Answer: b.

Remember: Overloading (same function name, different parameters) is decided by the compiler — so it's resolved at compile-time.

Next

Good job! 8/8

You obtain a badge:

OOP Challenger

Code Warrior

Problem Solver

Next

Good job! 7/8

You obtain a badge:

Problem Solver

OOP Challenger

Try again

Next

Good job! 6/8

You obtain a badge:

Problem Solver

Try again

Next

5/8

You’re progressing, strengthen your skills to climb higher.

Try again

Next

4/8

Don’t give up, review and try again. Every mistake is a step toward mastery.

Try again

Next

3/8

Don’t give up, review and try again. Every mistake is a step toward mastery.

Try again

Next

2/8

Don’t give up, review and try again. Every mistake is a step toward mastery.

Try again

Next

1/8

Don’t give up, review and try again. Every mistake is a step toward mastery.

Try again

Next

0/8

Don’t give up, review and try again. Every mistake is a step toward mastery.

Try again

Next

Post test Challenging

- A total of 8 questions
- You have 5:00 minutes total to complete the test.
- Each question must be answered within 30 seconds.
- You only have 3 lives, a wrong answer deducts one life.
- You only get 1 hint for the whole test (if used, it deducts 1 full point).
- Completing this test is a requirement to progress and earn your certificate.

Next

Are you ready?

Proceed

Post test

Challenging

5:00

1. What is wrong with this code?

```
class Animal {
public:
    void speak() { cout << "Animal"; }
};
class Dog : public Animal {
public:
    void speak() override { cout << "Dog"; }
};
```

a. override cannot be used without virtual in base class
b. Missing semicolon after class Dog
c. Cannot inherit without protected
d. No error

Hint

Answer: a.

Remember: In C++, a function must be declared virtual in the base class before a derived function can use override.

Next

Post test

Challenging

5:00

2. Predict the output:

```
class Base {
public:
    Base() { cout << "Base "; }
};
class Derived : public Base {
public:
    Derived() { cout << "Derived"; }
};
int main() {
    Derived d;
}
```

a. Base Derived
b. Derived Base
c. Derived only
d. Error

Hint

Answer: a.

Remember: When creating a Derived object, the Base constructor runs first, then the Derived constructor.

Next

Post test

Challenging

5:00

3. Why does this code fail to compile?

```
class Student {
private:
    int age;
};
int main() {
    Student s;
    s.age = 20;
}
```

a. age is private and cannot be accessed directly
b. Missing constructor
c. No main function return type
d. Needs public inheritance

Hint

Answer: a.

Remember: age is declared private, so it cannot be accessed outside the class.

Next

Post test

Challenging

5:00

4. Which is the best way to securely set a salary variable??
a. Make salary public
b. Use a setter function that validates input
c. Use protected keyword
d. Assign directly in main

Hint

Answer: b.

Remember: Encapsulation means using setter functions to validate and safely update private variables.

Next

Post test

Challenging

5:00

5. Predict the output:

```
class Shape {
public:
    virtual void draw() = 0;
};
class Circle : public Shape {
public:
    void draw() { cout << "Circle"; }
};
int main() {
    Shape* s = new Circle();
    s->draw();
}
```

a. Error
b. Circle
c. Shape
d. Nothing

Hint

Answer: b.

Remember: Shape is abstract, but Circle implements draw(). A base pointer to a derived object calls Circle's version.

Next

Post test

Challenging

5:00

6. What is wrong with this abstract class?

```
class Abstract {
public:
    virtual void display() = 0;
};
int main() {
    Abstract a;
}
```

a. Pure virtual functions need implementation
b. Cannot instantiate an abstract class
c. Missing semicolon
d. Needs inheritance

Hint

Answer: b.

Remember: Abstract classes cannot be instantiated directly. They require a derived class implementation.

Next

Post test

Challenging

5:00

7. Predict the output:

```
class Animal {
public:
    virtual void sound() { cout << "Animal"; }
};
class Cat : public Animal {
public:
    void sound() { cout << "Meow"; }
};
int main() {
    Animal a;
    a.sound();
}
```

a. Animal
b. Meow
c. Animal Meow
d. Meow

Hint

Answer: a.

Remember: a is a base object, not a pointer. Virtual dispatch does not apply here, it calls Animal's method.

Next

Post test

Challenging

5:00

8. Why is this code wrong?

```
class Math {
public:
    int add(int a, int b) { return a + b; }
    float add(float a, float b) { return a + b; }
};
int main() {
    Math m;
    cout << m.add(5, 2.5);
}
```

a. Ambiguity in overloaded methods
b. Must declare virtual functions
c. Constructors missing
d. add() cannot be overloaded

Hint

Answer: a.

Remember: Passing 5 (int) and 2.5 (float) creates ambiguity between int add(int,int) and float add(float,float).

Next

Good job! 8/8

You obtain a badge:

Code Gladiator

Persistent Debugger

Problem Solver

Next

Good job! 7/8

You obtain a badge:

Code Gladiator

Persistent Debugger

Try again

Next

Good job! 6/8

You obtain a badge:

Problem Solver

Try again

Next

5/8

You’re progressing, strengthen your skills to climb higher.

Try again

Next

4/8

Don’t give up, review and try again. Every mistake is a step toward mastery.

Try again

Next

3/8

Don’t give up, review and try again. Every mistake is a step toward mastery.

Try again

Next

2/8

Don’t give up, review and try again. Every mistake is a step toward mastery.

Try again

Next

1/8

Don’t give up, review and try again. Every mistake is a step toward mastery.

Try again

Next

0/8

Don’t give up, review and try again. Every mistake is a step toward mastery.

Try again

Next

Post test

Tough

4:00

- A total of 8 questions
- You have 4:00 minutes total to complete the post-test.
- Each question must be answered within 30 seconds.
- You only have 2 lives, a wrong answer deducts one life.
- You only get 1 hint for the whole test (if used, it deducts 1 full point).
- Completing this test is a requirement to progress and earn your certificate.

Next

Are you ready?

Proceed

Post test

Tough

4:00

1. Fill in the missing keyword to correctly implement inheritance.

```
class Vehicle {
public:
    void drive() { cout << "Driving"; }
};

class Car _____ Vehicle {
public:
    void honk() { cout << "Beep!"; }
};
```

a. inherits
b. extends
c. :
d. implements

Hint

Answer: : public

Hint: "Think of the C++ keyword used for inheritance."

Next

Post test

Tough

4:00

2. Fill in the missing keyword to protect data and access it safely.

```
class Account {
   _____:
      double balance;
   public:
      void setBalance(double b) { balance = b; }
};
```

a. public
b. private
c. protected
d. secure

Hint

Answer: private

Hint: "Encapsulation hides variables so they can't be accessed directly."

Next

Post test

Tough

4:00

3. Complete the abstract class definition.

```
class Shape {
public:
    virtual void draw() _____;
};
```

a. = default
b. = 1
c. = 0
d. abstract

Hint

Answer: = 0;

Hint: "A pure virtual function in C++ ends with __."

Next

Post test

Tough

4:00

4. Complete the abstract class definition.

```
class Animal {
public:
    _____ void speak() { cout << "Animal"; }
};

class Dog : public Animal {
public:
    void speak() _____ { cout << "Bark"; }
};
```

Hint

Answer: = virtual, override

Hint: "Runtime polymorphism in C++ requires two keywords: one in the base, one in the derived."

Choices for the first blank (Animal class):
a. static
b. virtual
c. override
d. final

Choices for the second blank (Dog class):
A. static
B. virtual
C. override
D. abstract

Next

Post test

Tough

4:00

5. Fill in the missing keyword to implement inheritance.

```
class Vehicle {
public:
    void drive() { cout << "Driving"; }
};

class Car _____ Vehicle {
public:
    void honk() { cout << "Beep!"; }
};
```

a. inherits
b. extends
c. :
d. implements

Hint

Answer: = : public

Hint: “C++ uses colon + access specifier, unlike Java.”

Next

Post test

Tough

4:00

6. Complete the constructor call for derived class.

```
class Base {
public:
    Base(int x) { cout << x; }
};
class Derived : public Base {
public:
    Derived(int y) : _____(y) { }
};
```

a. Derived
b. Base
c. super
d. parent

Hint

Answer: = : The derived class must call the Base constructor explicitly with Base(y).

Hint: "Which constructor is called when initializing derived class?"

Next

Post test

Tough

4:00

7. Fill in the missing keyword for data protection.

```
class Account {
   _____:
      double balance;
public:
   void setBalance(double b) { balance = b; }
};
```

a. public
b. private
c. protected
d. secure

Hint

Answer: = : private

Hint: “Which access specifier hides data completely?”

Next

Post test

Tough

4:00

8. Which keyword allows controlled inheritance access?

```
class Student {
protected:
    int grade;
public:
    void setGrade(int g) { grade = g; }
};
class CollegeStudent _____ Student {
public:
    void display() { cout << grade; }
};
```

a. private
b. public
c. protected
d. inherits

Hint

Answer: = : public

Hint: “In C++, inheritance is written with colon (:) and access specifier.”

Next

Good job! 8/8

You obtain a badge:

Grandmaster of code

Iron Coder

Unbreakable Spirit

Next

Good job! 7/8

You obtain a badge:

Iron Coder

Unbreakable Spirit

Try again

Next

Good job! 6/8

You obtain a badge:

Unbreakable Spirit

Try again

Next

5/8

You may have fallen, but your grit shines, rise and conquer again.

Try again

Next

4/8

You may have fallen, but your grit shines, rise and conquer again.

Try again

Next

3/8

You may have fallen, but your grit shines, rise and conquer again.

Try again

Next

2/8

You may have fallen, but your grit shines, rise and conquer again.

Try again

Next

1/8

You may have fallen, but your grit shines, rise and conquer again.

Try again

Next

0/8

You may have fallen, but your grit shines, rise and conquer again.

Try again

Next

Easy

Scores:

Pre-test	5/5
Exercise 1	3/5
Exercise 2	4/5
Post-test	10/10

© 2025 eMódulo | All Rights Reserved